

Doc. no.:	P0963R2
Date:	2024-05-14
Audience:	EWG, CWG
Reply-to:	Zhihao Yuan <zy@miator.net>

Structured binding declaration as a *condition*

- Structured binding declaration as a condition
 - Changes
 - Introduction
 - Motivation
 - Design Decisions
 - Unconditionally decompose
 - Testing is sequenced before decomposing
 - No underlying array object
 - Wording
 - Implementation
 - Acknowledgements
 - References

Changes

Since R1

- Sequence the test prior to decomposition; add discussion and adjust wording
- Refine wording

Since R0

- Rework the motivation
- Clarify that decomposition is sequenced before testing

Introduction

C++17 structured binding declaration is designed as a variant of variable declarations. As of today, it may appear as a statement on its own or as the declaration part of a range-based `for` loop. Meanwhile, the *condition* of an `if` statement may also be a variable declaration and can benefit from being a structured binding declaration. This paper proposes to allow structured binding declarations with initializers appearing in place of the *conditions* in `if`, `while`, `for`, and `switch` statements.

<i>simple-declaration</i>	<pre>auto [b, p] = ranges::mismatch(current, end, pbegin, pend);</pre>
<i>for-range-declaration</i>	<pre>for (auto [index, value] : views::enumerate(vec)) { println("{}: {}", index, value); ... }</pre>
<i>condition</i>	<pre>if (auto [to, ec] = std::to_chars(p, last, 42)) { auto s = std::string(p, to); ... }</pre>

Motivation

By design, structured binding is only about decomposition. The information of an object to be decomposed equals the information of all the components combined. However, after deploying structured bindings for a few years, it has been found that, in some scenarios, certain side information contributes to complexity if left out.

Scenario 1

The author sees a pattern that can be demonstrated using the following code snippet:

```
if (auto [first, last] = parse(begin(), end()); first != last) {
    // interpret [first, last) into a value
}
```

The idea is to split parsing and the action. Returning a pair of pointers makes it flexible to form different, windowed inputs.

However, if you wear glasses of "I did not write the code," the condition `first != last` doesn't say much. It's repetitive, opens the opportunity of being combined with other conditions, and can cause mistakes if comparing different pairs.

It would be nice if, when defining the intermediate type that carries the pairs to be decomposed, the condition can be baked into the type,

```
struct parse_window
{
    char const *first, *last;
    explicit operator bool() const noexcept { return first != last; }
};
```

and eliminates the need to maintain a convention:

```
if (auto [first, last] = parse(begin(), end())) {
    // interpret [first, last) into a value
}
```

In this example, **information about the condition is spread across the components**, and "how to form the condition" is not self-explanatory. If structured binding can channel this knowledge contextually, the library authors and the users may settle with a more solid pattern.

Scenario 2

Here is an updated example of using `<charconv>` in C++26 after adopting P2497^[1]:

```
if (auto result = std::to_chars(p, last, 42)) {
    auto [ptr, _] = result;
    // okay to proceed
} else {
    auto [ptr, ec] = result;
    // handle errors
}
```

We succeeded at restricting the variable to the minimal lexical scope where needed, but the code still struggled to implement what the users wanted to express.

The example can be a lot simpler if, when testing the `result` variable which has no role other than being decomposed later, the test is done as a part of decomposition without naming the intermediate `result` :

```
if (auto [ptr, ec] = std::to_chars(p, last, 42)) {  
    // okay to proceed  
} else {  
    // handle errors  
}
```

So, even when **a single component contains information about the condition** (`result.ec` in this example), people continue to be motivated to consolidate the knowledge of "how to test" into the complete object. But how to test when the complete object happens to be the underlying object of structured binding? The proposed feature answers the need.

Scenario 3

In an iterative solver, the code runs a primary solving step, like the following, in a loop. The call returns the state of the problem, decomposed into matrices and vectors:

```
auto [Ap, bp, x, y] = solve();
```

The solver must determine, right after the step, whether it gets an optimal solution. Mathematically, this can be done by evaluating one or more components like this:

```
if (is_optimal(x)) // scan the x vector  
    break;
```

But doing so may involve a linear algorithm or worse. Meanwhile, the `solve()` procedure may know whether the answer is optimal and save this information in the result as if it is cached. If the language allows retrieving this information, the following code can be terser and more efficient at the same time:

```
if (auto [Ap, bp, x, y] = solve()) // no need to scan x again  
    break;
```

In this example, **the information about the condition needs to be reconstructed from the components at a cost**. The complete object is an excellent place to cache this information but is not in a position to bring this redundant information into a separate component.

Scenario 4

Consider this example that uses the CTRE^[2] library:

```
if (auto [all, city, state, zip] = ctre::match<"(\\w+), (\\w+) (\\d+)">(s); all) {
    return location{city, state, zip};
}
```

It is surprising to see a regular expression that introduces three capture groups generating a result of four components unless the readers are already familiar with other Perl-like regex engines, which offer a "default" capture group to represent the entire match. Such a match group can be referred to as `\0` when performing regex-based substitution, which isn't what we're doing here (nor supported by CTRE as the time of writing, either).

It might be more WYSIWYG if, in the next generation of the API, three capture groups mean three components to extract:

```
if (auto [city, state, zip] = ctre2::match<"(\\w+), (\\w+) (\\d+)">(s)) {
    return location{city, state, zip};
}
```

In this example, if solely looking at the outcome, the information to be tested in the condition is not in the components. But still, when **all components but one have similar roles**, folding such a particular component into an implicit test well-suited for its role makes the code easier to understand.

Design Decisions

Unconditionally decompose

It is tempting to add extra semantics given the proposed syntax, such as conditionally evaluating the binding protocol after testing the underlying object:

```
auto consume_int() -> std::optional<int>;

if (auto [i] = consume_int()) { // let e be the underlying object
    // i = *e
} else {
    // *e is not evaluated
}
```

This idea turns `std::optional<T>` into a new kind of type that is "conditionally destructurable." Imagine this: if `[x]` can destructure `optional<T>`, then `[x, y]` won't destructure `optional<tuple<T, U>>`. The pattern matching proposal^[3] has better answers to these: `let ?x` and `let ?[x, y]`. With pattern matching, one can rewrite the hypothetical code snippet above as:

```
if (consume_int() match let ?i) {  
    // use(i)  
} else {  
    // has no value  
}
```

The idea of conditionally decomposing confuses sum types with product types; therefore, it is not included in this paper.

Testing is sequenced before decomposing

If decomposition is taken place unconditionally, when that happens becomes a question. Does it happen before evaluating the *condition* or after? The author's mental model for structured binding in *condition* is the following:

```
if (auto [a, b, c] = fn()) {  
    statements;  
}
```

is equivalent to

```
if (auto [a, b, c] = fn(); e) {  
    statements;  
}
```

where `e` is the underlying object of the structured binding declaration. If we go further and infer the semantics of the proposed control structure from the desugared form, the *condition* would be evaluated after decomposing the underlying object.

However, "design by desugaring" can generate suboptimal outcomes. A great example is the lifetime issue of range-based `for` loops. The latest refinement deviates their semantics from the desugared equivalent, but this change is what everybody wants.^[4]

In the context of this paper, as Tim Song pointed out, users expect to test and decompose a subrange into iterators without naming the range,

```

if (auto [b, e] = compute_some_subrange())
{
    // ...
}

```

but this will not work under the aforementioned "desugaring" model if the bindings refer to move-only iterators, as in effect, we will be testing moved-from objects.

```

auto r = compute_some_subrange();
if (auto [b, e] = std::move(r); r) // approximately
{
    // ...
}

```

Compiler Explorer

The following code ([3h74oq8zW](https://gcc.godbolt.org/z/3h74oq8zW) (<https://gcc.godbolt.org/z/3h74oq8zW>)) incurs undefined behavior in the compiler that implements the R1 semantics of this paper:

```

std::generator<int> f()
{
    co_yield 1;
    co_yield 2;
}

int main()
{
    if (auto g = f(); auto [b, e] = std::ranges::subrange{g})
    {
        return 0;
    }
}

```

We could imagine that what the users are looking for is a hypothetical `if` statement in which the first declaration in parenthesis is interpreted as the *condition* and the second as the *init-statement*:

```

if (auto e = fn(); auto [a, b, c] = e) {
    statements;
}

```

This makes sense because contextually converting the underlying object of structured binding to `bool` is a side channel to pass information. We could mandate extracting this information first when doing so is motivated, as well as the order of extracting the other

pieces of information^[5]. This paper proposes evaluating the *condition* before initializing the bindings.

No underlying array object

It is worthwhile to figure out what array decomposition does in a *condition*. The *condition* forbids declaring arrays, so this paper neither allows decomposing arrays. However, the *condition* accepts array references, which always evaluate to `true`, which is also unchanged in this paper. The following works with the proposed change:

```
if (auto& [a, b, c] = "ht")
    // true branch is always taken
```

Decomposing arrays in *conditions* is very unmotivated.

Wording

The wording is relative to N4981.

Extend the grammar in [stmt.stmt]/1 (<https://eel.is/c++draft/stmt.stmt#stmt.pre-1>) as follows:

condition:
expression
attribute-specifier-seq_{opt} decl-specifier-seq declarator brace-or-equal-initializer
structured-binding-declaration brace-or-equal-initializer

Modify [stmt.stmt]/4 (<https://eel.is/c++draft/stmt.stmt#stmt.pre-4>) as follows:

The rules for *conditions* apply both to *selection-statements* ([stmt.select] (<https://eel.is/c++draft/stmt.stmt#stmt.select>)) and to the *for* and *while* statements ([stmt.iter] (<https://eel.is/c++draft/stmt.stmt#stmt.iter>)). If a *structured-binding-declaration* appears in a *condition*, the *condition* is a structured binding declaration ([dcl.struct.bind] (<https://eel.is/c++draft/dcl.struct.bind>)). The *brace-or-equal-initializer* shall be of the form "= *assignment-expression*" or "{ *assignment-expression* }", where the *assignment-expression* shall not be of array type if no *ref-qualifier* is present in the *structured-binding-declaration*. A *condition* that is ~~not~~neither an *expression* nor a structured binding declaration is a declaration ([dcl.dcl] (<https://eel.is/c++draft/dcl.dcl>)). The *declarator* shall not specify a function or an array. The *decl-specifier-seq* shall not define a class or enumeration. If the *auto type-specifier* appears in the *decl-specifier-seq*, the type of the identifier being declared is deduced from the initializer as described in [dcl.spec.auto] (<https://eel.is/c++draft/dcl.spec.auto>).

Insert a paragraph between [stmt.stmt]/4 (<https://eel.is/c++draft/stmt.stmt#stmt.pre-4>) and [stmt.stmt]/5 (<https://eel.is/c++draft/stmt.stmt#stmt.pre-5>):

The *decision variable* of a *condition* that is an initialized declaration other than structured binding declaration is the declared variable. The decision variable of a *condition* that is a structured binding declaration is described in [dcl.struct.bind].
(<https://eel.is/c++draft/dcl.struct.bind>).

Edit the original [stmt.stmt]/5 (<https://eel.is/c++draft/stmt.stmt#stmt.pre-5>) as follows:

The value of a *condition* that is an initialized declaration in a statement other than a `switch` statement is the value of the ~~declared~~decision variable contextually converted to `bool` ([conv] (https://eel.is/c++draft/conv#def:conversion,contextual_to_bool)). If that conversion is ill-formed, the program is ill-formed. The value of a *condition* that is an expression is the value of the expression, contextually converted to `bool` for statements other than `switch`; if that conversion is ill-formed, the program is ill-formed. The value of the condition will be referred to as simply "the condition" where the usage is unambiguous.

Edit [stmt.switch]/2 (<https://eel.is/c++draft/stmt.switch>) as follows:

The value of a *condition* that is an initialized declaration is the value of the ~~declared~~decision variable, or the value of the *expression* otherwise. The value of the condition shall be of integral type, enumeration type, or class type. If of class type, the condition is contextually implicitly converted ([conv] (https://eel.is/c++draft/conv#def:contextually_implicitly_converted)) to an integral or enumeration type. If the (possibly converted) type is subject to integral promotions ([conv.prom] (<https://eel.is/c++draft/conv.prom>))), the condition is converted to the promoted type. [...]

Insert a paragraph between [dcl.struct.bind]/1 (<https://eel.is/c++draft/dcl.struct.bind#1>) and [dcl.struct.bind]/2 (<https://eel.is/c++draft/dcl.struct.bind#2>):

If a structured binding declaration appears as a *condition*, the decision variable ([stmt.pre] (<https://eel.is/c++draft/stmt.pre>)) of the condition is `e`.

[Drafting note: The wording to be added by CWG2867 is highlighted . —end note]

Modify the original [dcl.struct.bind]/4 (<https://eel.is/c++draft/dcl.struct.bind#4>) as follows:

[...], otherwise, variables are introduced with unique names r_i as follows:

$$S \cup_i r_i = \text{initializer};$$

Each v_i is the name of an lvalue of type T_i that refers to the object bound to r_i ; the referenced type is T_i . The initialization of e and any conversion of e considered as a decision variable (`[stmt.stmt]` (<https://eel.is/c++draft/stmt.stmt>)). is sequenced before the initialization of any r_i . The initialization of r_i is sequenced before the initialization of r_j if $i < j$.

Implementation

R1 semantics has been shipped in Clang since 6.0.0, guarded by `-Wbinding-in-condition :`
Compiler Explorer
`b64x65716` (<https://godbolt.org/z/b64x65716>); R2 has not been implemented.

Acknowledgements

Thank Richard Smith for encouraging the work and Hana Dusíková for providing motivating examples. Thank Tim Song for additional examples that suggest the revised semantics. Thank Jens Maurer for the wording review.

References

1. Wakely, Jonathan. P2497R0 *Testing for success or failure of `<charconv>` functions*.
<https://wg21.link/p2497r0> (<https://wg21.link/p2497r0>) 
2. Dusíková, Hana. P1433R0 *Compile Time Regular Expressions*.
<https://wg21.link/p1433r0> (<https://wg21.link/p1433r0>) 
3. Park, Michael. P2688R1 *Pattern Matching: `match` Expression*.
<https://wg21.link/p2688r1> (<https://wg21.link/p2688r1>) 
4. Josuttis, Nicolai, et al. P2644R1 *Final Fix of Broken Range-based for Loop, Rev 1*.
<https://wg21.link/p2644r1> (<https://wg21.link/p2644r1>) 

5. Smith, Richard. CWG2867 *Order of initialization for structured bindings*.
<https://cplusplus.github.io/CWG/issues/2867.html>
(<https://cplusplus.github.io/CWG/issues/2867.html>) 